

Verificação, Validação e Teste de Software (VVT) 2026.1

Week 5 - Testes Funcionais

Prof. Dr. Awdren de Lima Fontão



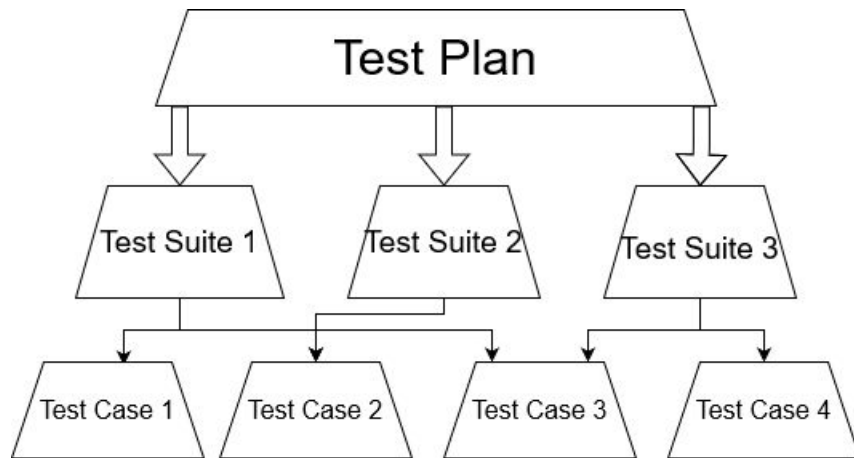
UNIVERSIDADE
FEDERAL DE
MATO GROSSO DO SUL



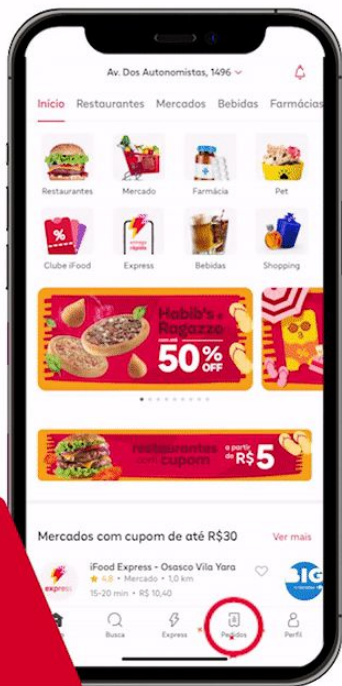
Por que suíte de teste?

Usar o termo “suíte” comunica que:

- Há **organização intencional** (por funcionalidades, objetivos, criticidade).
- Os testes **se complementam ou se relacionam**.
- O grupo de testes pode ser **executado como uma unidade**.
- Existem **propriedades comuns** (como entrada, objetivo, ou contexto de execução).



Meu pedido veio errado.
O que eu faço?



Você foi contratado(a) como engenheiro de testes pelo iFood para participar da homologação de uma nova versão do aplicativo.

Essa versão inclui ajustes nas regras de frete grátis, limites de pedidos e validação de cadastro de usuários. Seu time precisa validar se tudo está funcionando corretamente do ponto de vista do usuário final, ou seja, sem olhar o código.

Você terá acesso apenas às especificações fornecidas pelo time de produto.

A equipe de desenvolvimento quer sua ajuda para identificar possíveis falhas antes que o app chegue a milhões de usuários.

Missão (grupos de 3 a 5 pessoas)

Com base nos requisitos e US:

- Crie pelo menos uma situação de teste com comportamento incorreto (falha realista).

Descreva:

- O que o cliente tentou fazer
- O que o sistema fez
- O que deveria ter feito
- Como vocês testariam esse comportamento

Registre os dados de entrada, a saída esperada e a justificativa do teste.

 Dica: pense como um(a) cliente insatisfeito(a) ou como alguém tentando "burlar" o sistema.

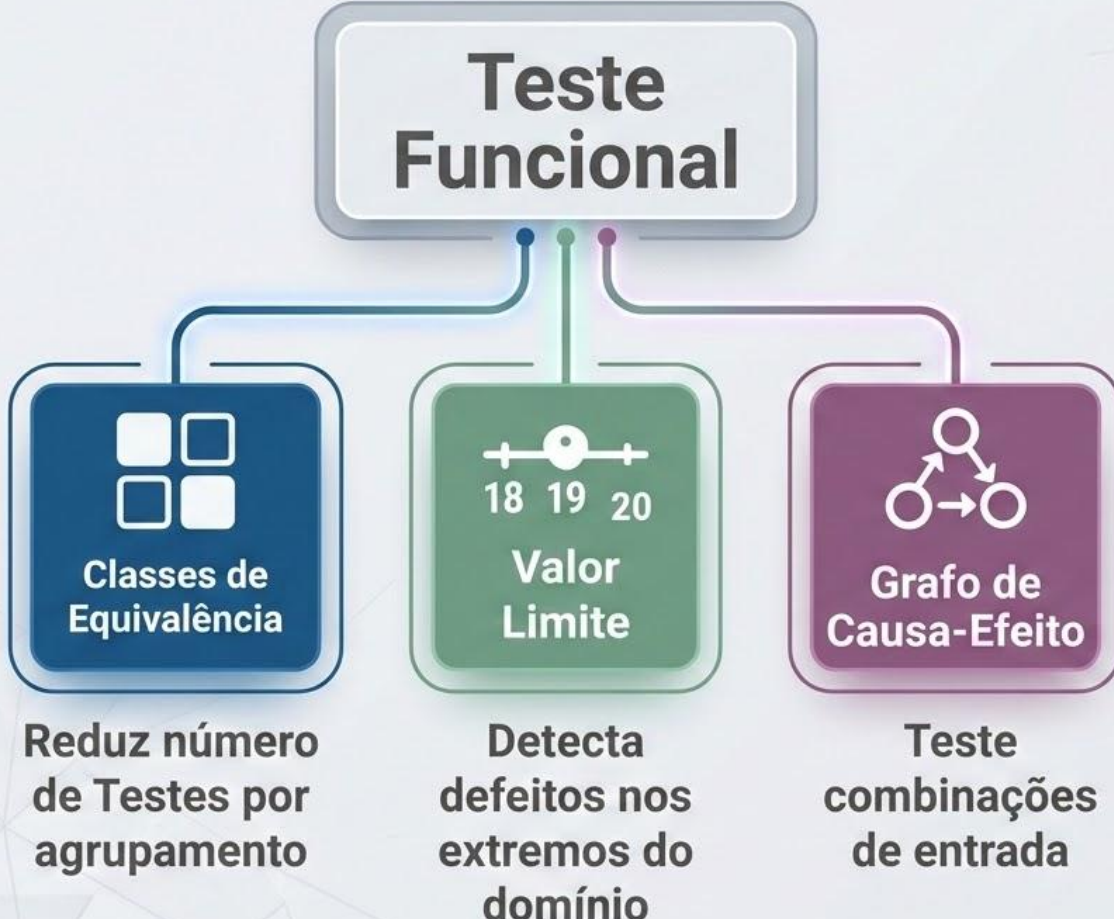
Exemplo

Requisito: Valor do pedido deve ser entre R\$10 e R\$500

Situação criada pelo grupo:

- Cliente adicionou um item de R\$7 ao carrinho. O app deixou seguir para o pagamento e confirmou o pedido;
- Deveria ter aparecido uma mensagem impedindo a finalização;
- Vamos testar com valores como R\$9, R\$10, R\$500 e R\$501 para verificar o comportamento nos limites.

Teste Funcional



```
graph TD; A[Teste Funcional] --> B[Classes de Equivalência]; A --> C[Valor Limite]; A --> D[Grafo de Causa-Efeito];
```



Classes de
Equivalência

Reduz número
de Testes por
agrupamento



Valor
Limite

Detecta
defeitos nos
extremos do
domínio



Grafo de
Causa-Efeito

Teste
combinações
de entrada

São **heurísticas
iniciais**, não regras
universais.

Técnicas (guiar como pensar) para Teste de Software

1.



Funcional (ou caixa preta/caixa fechada)

Baseada na especificação.
Aborda o software macroscopicamente.

2.



Estrutural (ou caixa branca/caixa aberta)

Baseada na implementação.
Aborda a estrutura interna.

3.



Baseada em erros

Baseada na experiência e intuição sobre falhas comuns.

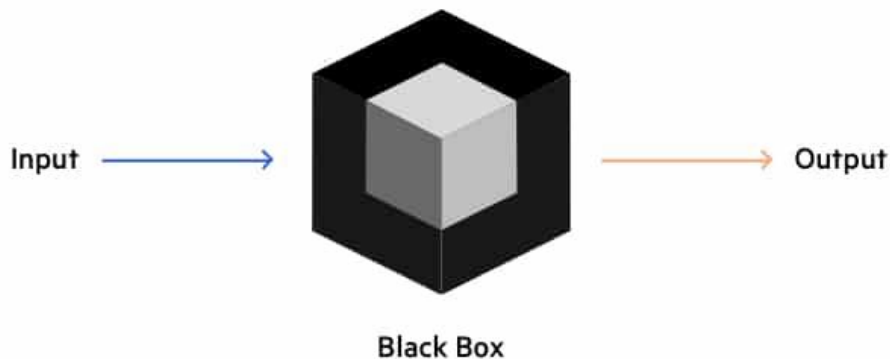
A questão não está em qual delas utilizar e sim como combiná-las de forma a maximizar os benefícios das atividades de teste!

Técnica Funcional



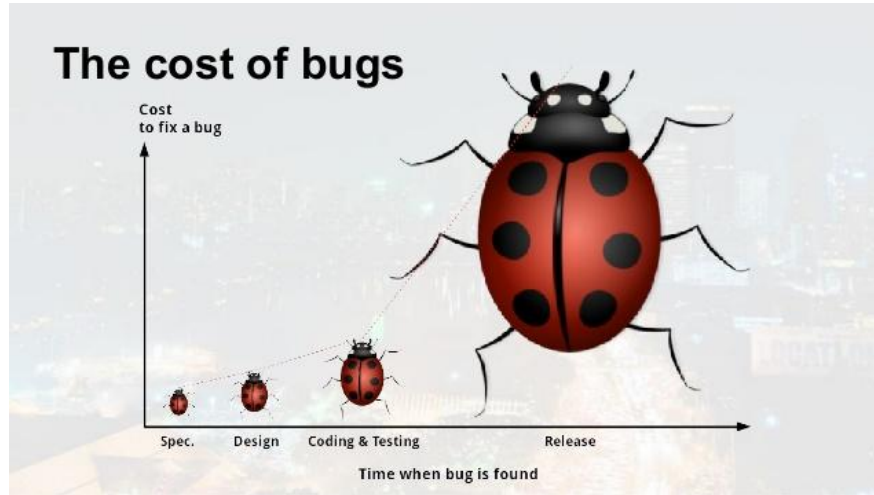
- Baseia-se na especificação do software para derivar os requisitos de teste;
- Aborda o software de um ponto de vista macroscópico;
- Envolve dois passos principais:
 - identificar as funções que o software deve realizar (especificação dos requisitos, casos de uso);
 - criar casos de teste capazes de verificar se essas funções estão sendo executadas corretamente

Teste funcional é uma técnica utilizada para se projetarem casos de teste na qual o programa ou sistema é considerado uma caixa-preta.



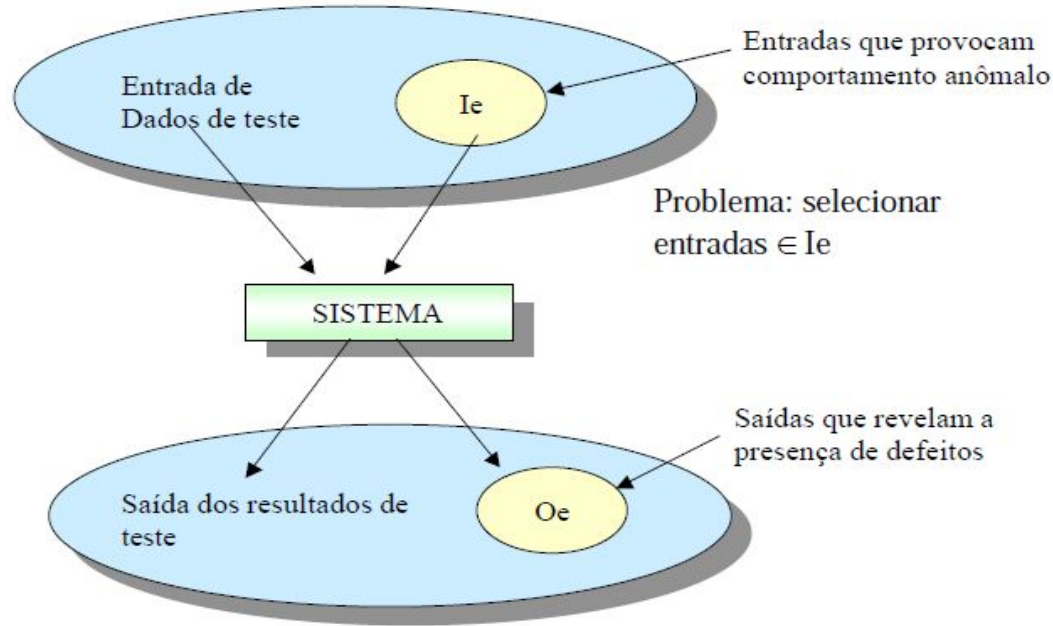
Nessa técnica os detalhes de implementação não são considerados e o software é avaliado segundo o ponto de vista do usuário.

Teste exaustivo?



Em princípio, o teste funcional pode detectar todos os defeitos, submetendo o programa ou sistema a todas as possíveis entradas, o que é denominado teste exaustivo.

Técnica Funcional



O domínio de entrada pode ser infinito ou muito grande, de modo a tornar o tempo da atividade de teste inviável, fazendo com que essa alternativa se torne impraticável.

Essa limitação da atividade de teste, que não permite afirmar, em geral, que o programa esteja correto, fez com que fossem definidas as técnicas de teste e os diversos critérios pertencentes a cada uma delas.

Técnica Funcional

Problema:

Dificuldade em quantificar a atividade de teste – não se pode garantir que partes essenciais ou críticas do software foram executadas (exercitadas)

Critérios dessa técnica baseiam-se somente na especificação de requisitos para derivar os requisitos de testes.

Desvantagens do Teste Caixa Preta

- Dependente de uma boa especificação de requisitos o que, em geral, não é bem feito.
- Não é possível garantir que partes essenciais ou críticas do software sejam executadas.
- Para encontrar todos os defeitos utilizando teste caixa preta é necessário o teste exaustivo.
 - Como testar todas as possíveis entradas para um compilador? (Myers, 1979).

Vantagens do Teste Caixa Preta

- Independente do paradigma de programação utilizado.
- Eficaz em detectar determinados tipos de erros.
 - Funcionalidade ausente, por exemplo

Precisamos:

1. Explorar determinados tipos de defeitos;
2. Estabelecer requisitos de testes para os quais valores específicos do domínio de entrada devem ser definidos;
3. Temos que exercitar os valores específicos.

Como todos os critérios (regra específica -> técnica bem aplicada) da técnica funcional baseiam-se apenas na especificação do produto testado...

- a qualidade de tais critérios depende fortemente da existência de uma boa especificação de requisitos.

Especificações ausentes ou mesmo incompletas tornarão difícil a aplicação dos critérios funcionais.

Além disso, tais critérios também apresentam a limitação de não garantir que partes essenciais ou críticas do produto em teste sejam exercitadas.



Os critérios funcionais podem ser aplicados em todas as fases de testes e em produtos desenvolvidos com qualquer paradigma de programação, pois não levam em consideração os detalhes de implementação.



Particionamento de Equivalência

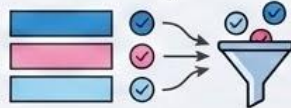
Particionamento de Equivalência

1. Divisão do Domínio

O testador selecionará CTs das partições -> isso deve dar uma boa representação de todo domínio.



2. Seleção de CTs



3. Foco Direcionado

Domínio de entrada **NA DIREÇÃO** de subdomínios de interesse.



Particionamento em Classes de Equivalência

- Classes de equivalência é um critério de teste funcional que divide os dados de entrada em grupos com comportamento esperado semelhante, permitindo selecionar poucos casos representativos em vez de testar todos os valores possíveis.
- Exemplo: sistema de recursos humanos – empregar pessoas com base na idade

0 – 16	Não empregar.
16 – 18	Pode ser empregado tempo parcial.
18 – 55	Pode ser empregado tempo integral.
55 – 99	Não empregar.

Particionamento em Classes de Equivalência

- O módulo deveria ser testado considerando as idades: 0, 1, 2, 3, 4, 5, 6, 7, 8, ..., 90, 91, 92, 93, 94, 95, 96, 97, 98, 99?
- Considere que o módulo que resolve o problema anterior tenha sido implementado como ilustrado abaixo:

```
1  if (idade == 0)  empregar = "NAO";
2  if (idade == 1)  empregar = "NAO";
3  ...
4  if (idade == 15) empregar = "NAO";
5  if (idade == 16) empregar = "PAR";
6  if (idade == 17) empregar = "PAR";
7  if (idade == 18) empregar = "INT";
8  if (idade == 19) empregar = "INT";
9  ...
10 if (idade == 53) empregar = "INT";
11 if (idade == 54) empregar = "INT";
12 if (idade == 55) empregar = "NAO";
13 if (idade == 56) empregar = "NAO";
14 ...
15 if (idade == 98) empregar = "NAO";
16 if (idade == 99) empregar = "NAO";
```

Particionamento em Classes de Equivalência

- Se quisermos alta confiança nessa implementação específica, seria necessário testar mais valores do domínio
- Caso haja tempo suficiente, esse é o melhor teste a ser realizado.
- O problema é que da forma como o código foi implementado, a execução de um dado caso de teste não diz nada a respeito da execução do próximo.

Particionamento em Classes de Equivalência

- Agora considere essa outra implementação (bem melhor) do mesmo problema:

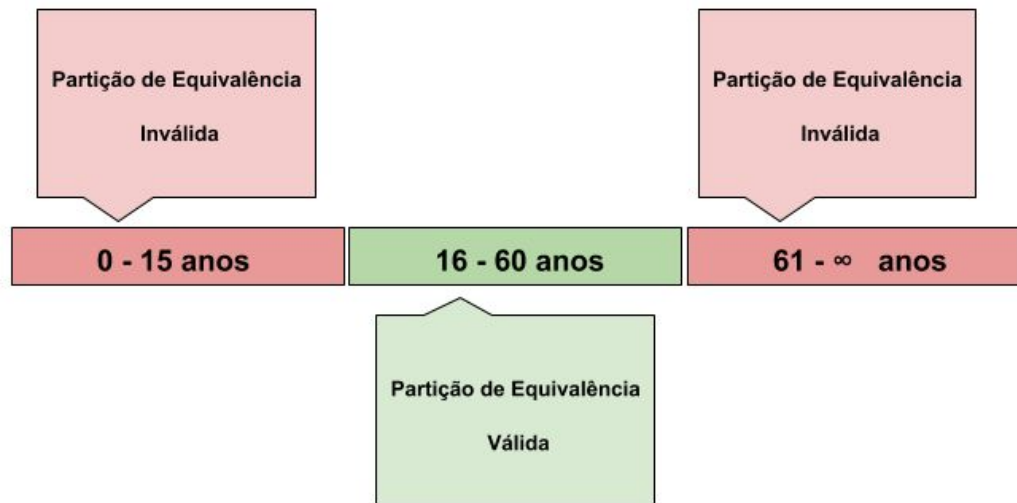
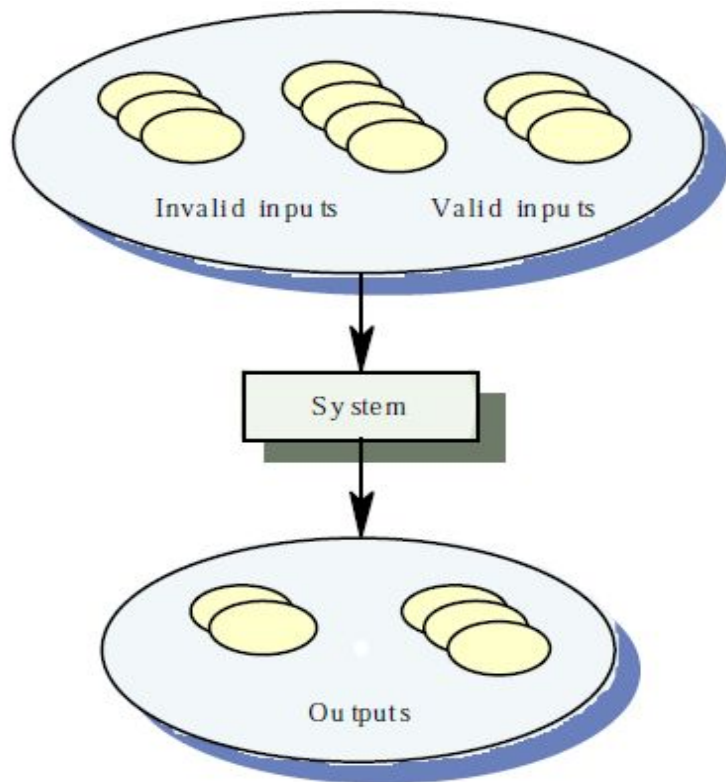
```
1  if (idade >= 0 && idade <= 16)
2      empregar = "NAO";
3  if (idade >= 16 && idade <= 18)
4      empregar = "PAR";
5  if (idade >= 18 && idade <= 55)
6      empregar = "INT";
7  if (idade >= 55 && idade <= 99)
8      empregar = "NAO";
```

- Dada essa implementação, fica claro que não é necessário testar para todos os valores $0, 1, 2, \dots, 14, 15$ e 16 , por exemplo.
- Apenas um valor precisa ser testado.

Particionamento em Classes de Equivalência

- Qualquer valor dentro do intervalo tem a mesma importância, ou seja, qualquer valor escolhido é adequado.
- O mesmo se aplica para os demais intervalos de dados
- Tais intervalos determinam o que é chamado de classe de equivalência
- Qualquer valor no intervalo de uma classe é considerado equivalente em termos de teste. Assim sendo:
 - Assume-se que valores da mesma classe tendem a produzir comportamento semelhante; por isso, escolhe-se um representante da classe.

Particionamento em Classes de Equivalência



Particionamento em Classes de Equivalência

Uma heurística comum é definir:

1. Se a condição de entrada especifica um intervalo de valores, define-se: Uma classe válida e duas inválidas;
2. Se a condição de entrada especifica uma quantidade de valores: Uma classe válida e duas inválidas;
3. Se a condição de entrada especifica um conjunto determinado de valores: Uma classe válida e outra inválida para cada um dos valores;
4. Se a condição de entrada especifica que “deve ser desta forma”: Uma classe válida e outra inválida.

Se a condição de entrada especifica um intervalo de valores

Requisito: O valor do pedido deve estar entre R\$10 e R\$500.

Classes de equivalência:

- Válida: R\$10 a R\$500
- Inválida 1: Menor que R\$10 (ex: R\$5)
- Inválida 2: Maior que R\$500 (ex: R\$600)

Se a condição de entrada especifica uma quantidade de itens (o número de elementos e não o valor)

Requisito: O cliente pode pedir de 1 a 10 pratos por pedido.

Classes de equivalência:

- Válida: 1 a 10 pratos (ex: 5)
- Inválida 1: Menor que 1 (ex: 0)
- Inválida 2: Maior que 10 (ex: 15)

Se a condição de entrada especifica um conjunto determinado de valores

Requisito: A forma de pagamento deve ser *PIX*, *Cartão* ou *Dinheiro*.

Classes de equivalência:

- Válida: PIX, Cartão, Dinheiro
- Inválida: Qualquer outro valor (ex: “Boleto”, “Transferência”, “Bitcoin”)

Se a condição de entrada especifica que “deve ser desta forma”

Requisito: O e-mail deve conter o caractere @.

Classes de equivalência:

- Válida: Contém @ (ex: cliente@delivery.com)
- Inválida: Não contém @ (ex: clientedelivery.com)

Exercício *Identifier*

- Especificação (extraído de Maldonado *et al.* (2004)):
 - O programa deve determinar se um identificador é válido ou não em Silly Pascal (uma variante do Pascal).
 - Um identificador válido deve começar com uma letra e conter apenas letras ou dígitos.
 - Além disso, deve ter no mínimo um caractere e no máximo seis caracteres de comprimento.

Exemplo:

```
abc12 (válido);      1soma (inválido);  
cont*1 (inválido);  a123456 (inválido)
```

Exercício *Identifier*

Classes de Equivalência:

Condições de Entrada	Classes Válidas	Classes Inválidas	
Tamanho t do identificador	$1 \leq t \leq 6$ (1)	$t < 1$ (2)	$t > 6$ (3)
Primeiro caractere c é uma letra	Sim (4)	Não (5)	
Só contém caracteres válidos	Sim (6)	Não (7)	

Exemplo de Conjunto de Teste:

$T_0 = \{(a1, \text{Valid}), ("", \text{Invalid}), (A1b2C3d, \text{Invalid}), (2B3, \text{Invalid}), (Z\#12, \text{Invalid})\}$

Cenário de Delivery de comida

Sistema: Aplicativo de delivery de comida

Requisito: O valor do pedido deve ser entre R\$10 e R\$500.

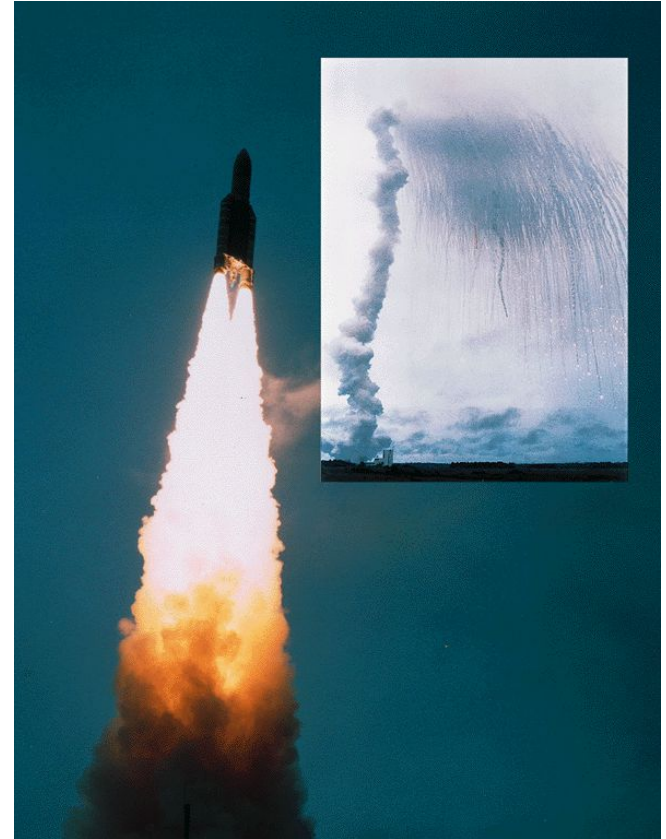
Classes de equivalência:

- **Válida:** R\$10 a R\$500 (ex: R\$50)
- **Inválidas:** abaixo de R\$10 (ex: R\$5) e acima de R\$500 (ex: R\$600)

Quantos CTs eu gero?

Análise do Valor Limite

- Junho de 1996: o CNES (Centro Nacional de Estudos Espaciais) e a ESA (Agência Espacial Européia) lançaram o foguete Ariane 5.
- Segundo consta nos laudos, uma falha de *overflow* causou a autodestruição do foguete após 37 segundos da decolagem, ocasionando um prejuízo de aproximadamente 500 milhões de dólares



Análise do Valor Limite

1

Por que usar AVL?



Em muitos sistemas, uma quantidade significativa de defeitos tende a se concentrar nos limites do domínio de entrada, e não apenas em valores intermediários. Por isso, é importante testar cuidadosamente esses pontos de transição.



2

O que é?

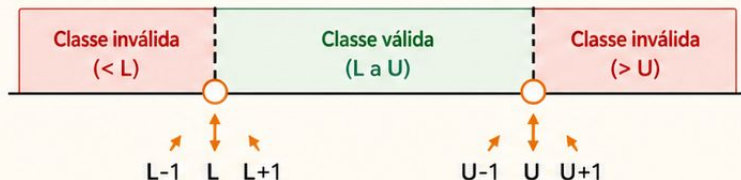
A Análise de Valor Limite (AVL) é uma técnica de teste funcional usada para projetar casos de teste com foco nos valores situados nos **limites das entradas** válidas e inválidas.



3

Relação com classes de equivalência

Essa técnica complementa o particionamento em classes de equivalência, pois, após identificar as classes, o testador seleciona valores localizados nas suas fronteiras, onde a ocorrência de falhas é mais provável.



4

Precisão dos testes

De acordo com as características do software e do tipo de dado tratado, é possível definir o nível de precisão adequado para os testes, considerando, por exemplo, valores inteiros, decimais, datas, tamanhos ou faixas permitidas.

123

Números inteiros

12,34

Valores decimais



Datas



Tamanhos



Faixas / intervalos



Ideia central: após identificar as classes de equivalência, **teste os valores nas fronteiras — imediatamente abaixo, no limite e imediatamente acima.**

De acordo com Meyers *et al.*, 2024, a experiência mostra que casos de teste que exploram condições limites têm maior probabilidade de encontrar defeitos.

Tais condições correspondem a valores que estão exatamente sobre ou imediatamente acima ou abaixo dos limitantes das classes de equivalência.

Análise do Valor Limite

$0 \leq idade \leq 15$	Não empregar.
$16 \leq idade \leq 17$	Pode ser empregado tempo parcial.
$18 \leq idade \leq 54$	Pode ser empregado tempo integral.
$55 \leq idade \leq 99$	Não empregar.

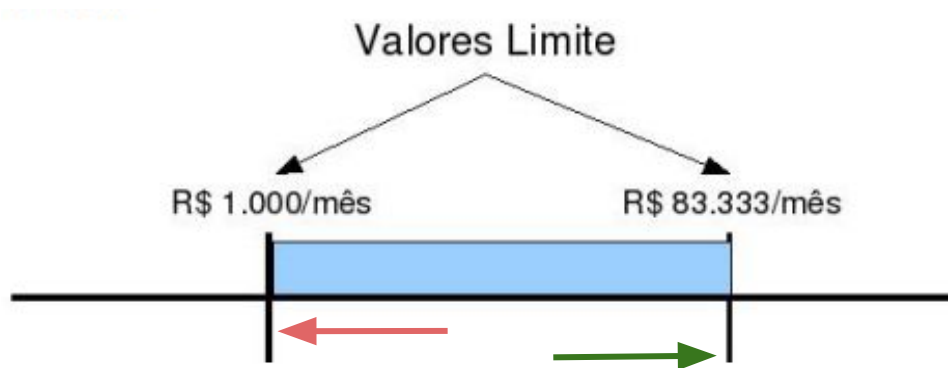
- Um dos critérios de teste mais básico que existe;
- Auxilia na seleção de um pequeno subconjunto de casos de teste que mantém boa cobertura do domínio de entrada e dos comportamentos especificados.
- Exemplo (mesmo usado para particionamento por equivalência).

VALORES LIMITES:

- REGRA 1 (não empregar): Inferior (-1, **0**, 1) e Superior (14, **15** e 16)
- REGRA 2 (parcial): Inferior (15, **16**, 17) e Superior (16, **17** e 18)
- REGRA 3 (integral): Inferior (17, **18**, 19) e Superior (53, **54** e 55)
- REGRA 4 (não empregar): Inferior (54, **55**, 56) e Superior (98, **99**, 100)

Definição das Classes

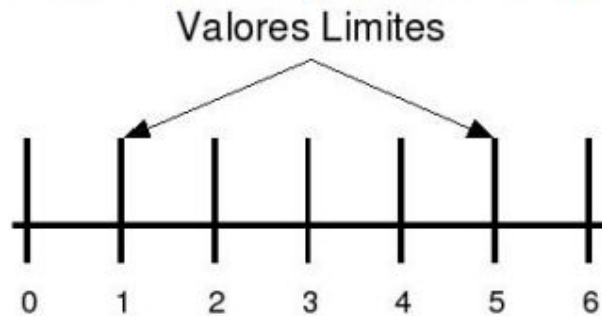
- Mais adequado para entradas que apresentam valores contínuos



- Candidatos a Dados de teste
 - Limite inferior: {\$999;\$1.000;\$1.001}
 - Limite superior: {\$83.332;\$83.333;\$83.334}

Definição das Classes

- Intervalo de dados discretos (hipotecas de 1 a 5 casas):



- Candidatos a Dados de teste
 - Limite inferior: {0; 1; 2}
 - Limite superior: {4; 5; 6}

Análise de Valor Limite: Delivery

Regra de Negócio: O valor do pedido deve estar entre R\$10 e R\$500

Limite Inferior

R\$10

Limite Superior

R\$500

Cenário Inferior

- **R\$9** (Inválido)
- **R\$10** (Válido)
- **R\$11** (Válido)

Cenário Superior

- **R\$499** (Válido)
- **R\$500** (Válido)
- **R\$501** (Inválido)

Quantos CTs (Casos de Teste) são necessários?

Análise do Valor Limite: Limitações

O Particionamento de Equivalência e a Análise de Valor Limite são excelentes para **valores individuais** (ex: idade, valor do pedido).



Falta de Combinação

Essas técnicas **não consideram combinações** de variáveis de entrada.



Interação Oculta

Se duas ou mais variáveis interagem para causar um defeito, essas técnicas **não detectarão o erro**, pois testam cada entrada de forma isolada.

Solução recomendada: Grafo de Causa-Efeito ou Tabelas de Decisão para testes combinatórios.

Exemplo no app de delivery

Regra: Frete grátis somente se o pedido for acima de R\$100 e o pagamento for online.

Problema:

- Testar só o valor do pedido com partições ou limites não garante que o método de pagamento foi corretamente avaliado.
- Testar só o método de pagamento também não basta.

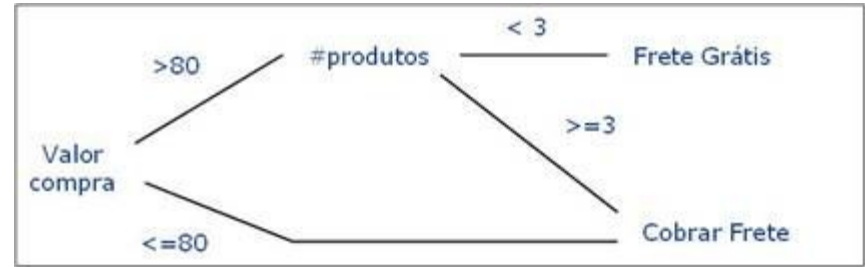
Solução:

É preciso testar a combinação entre os dois fatores (ex: valor + forma de pagamento).

Isso é feito com Grafo de Causa-Efeito ou Tabelas de Decisão, que permitem considerar várias entradas simultaneamente.

Grafo de Causa-Efeito

Grafo de Causa-Efeito



Usado para analisar **combinações de condições de entrada**.

Modela relações entre:

- **causas**: condições de entrada
- **efeitos**: saídas ou ações do sistema

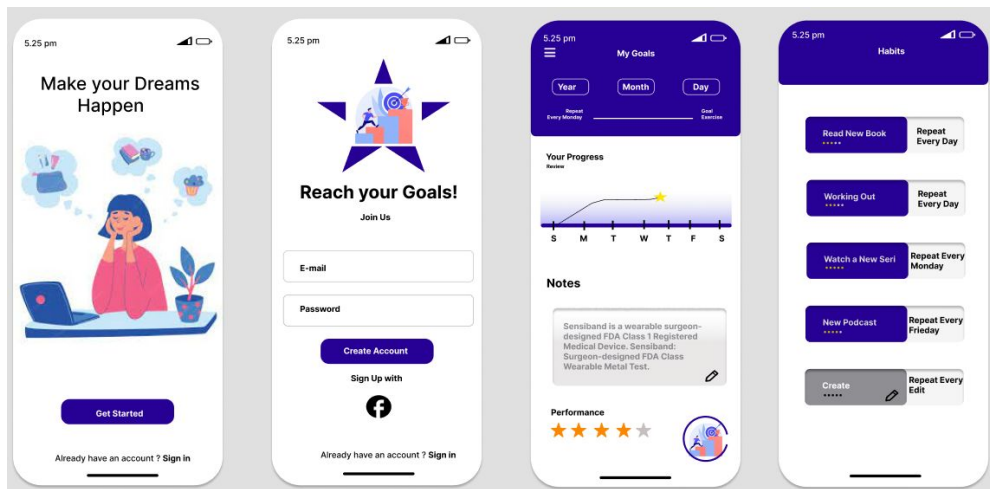
Ajuda a identificar **combinações logicamente relevantes** para os casos de teste

O grafo de causa-efeito pode ser convertido em uma tabela de decisão.

É especialmente útil quando o comportamento do sistema depende de **regras lógicas** e da combinação entre múltiplas entradas.

Tem conexão com algo conhecido?

Dica: IHC...



Grafo de Causa-Efeito:

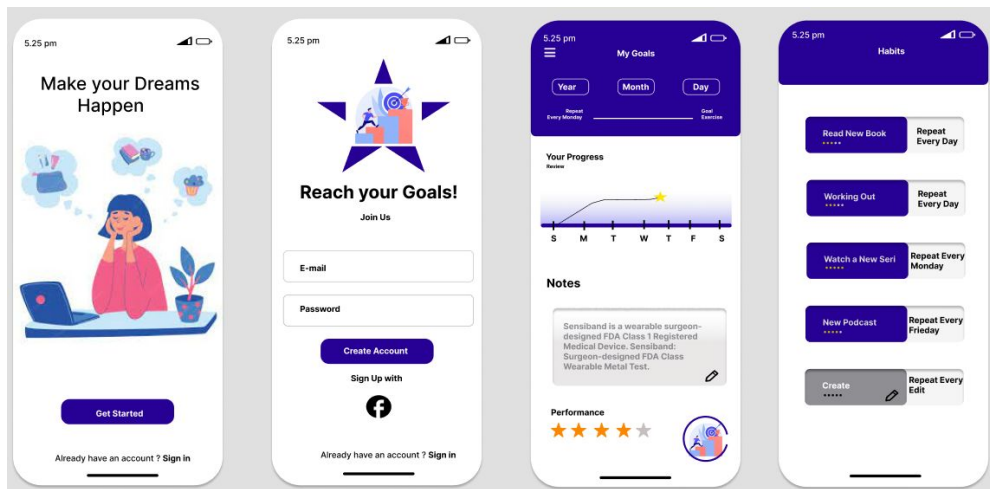
- Modela **condições de entrada (causas)** e **ações esperadas (efeitos)**;
- Usado para derivar **combinações lógicas de casos de teste**;
- Traduzido em **tabela de decisão** para gerar **casos de teste funcionais**.

Protótipos no Figma:

- Representam a **interface** e o **fluxo visual** da aplicação. Mostram **como o sistema deve reagir** às ações do usuário.

Tem conexão com algo conhecido?

Dica: IHC...



1. Ações do usuário no Figma = Causas no grafo

Exemplo:

- "Usuário clica em 'Finalizar pedido' com e-mail inválido": causa
- "Usuário seleciona pagamento em dinheiro com pedido acima de R\$100": causa

2. Respostas da interface no Figma = Efeitos no grafo

Exemplo:

- Exibir mensagem de erro;
- Liberar botão de frete grátis;
- Redirecionar para página de confirmação

Grafo de Causa-Efeito

Para cada módulo, Causas (condições de entrada) e efeitos (ações realizadas às diferentes condições de entrada) são relacionados, atribuindo-se um identificador para cada um.



Em seguida, um grafo de causa-efeito (árvore de decisão) é desenhado.



Neste ponto, transforma-se o grafo numa tabela de decisão.



As regras da tabela de decisão são, então, convertidas em casos de teste.

Grafo de Causa-Efeito

- Definições
 - Causas: condições de entrada (valor lógico)
 - Efeitos: ações realizadas em resposta às diferentes condições de entrada
- Exemplo:
 - Suponha que uma companhia de seguros oferece desconto especial para motoristas que são casados e/ou com bom desempenho escolar.

Condições	Entrada 1	Entrada 2	Entrada 3	Entrada 4
Casado	SIM	SIM	NÃO	NÃO
Bom desempenho escolar	SIM	NÃO	SIM	NÃO

Grafo de Causa-Efeito

- Exemplo:
 - Assim, têm-se duas condições, cada uma com dois possíveis valores, resultando em $2^2 = 4$ regras.
 - O testador deve verificar se todas as combinações foram definidas.

Condições	Entrada 1	Entrada 2	Entrada 3	Entrada 4
Casado	SIM	SIM	NÃO	NÃO
Bom desempenho escolar	SIM	NÃO	SIM	NÃO

Grafo de Causa-Efeito

- Exemplo:
 - Na verdade, mais de uma ação pode estar associada com determinada regra.

Condições	Entrada 1	Entrada 2	Entrada 3	Entrada 4
Casado	SIM	SIM	NÃO	NÃO
Bom desempenho escolar	SIM	NÃO	SIM	NÃO
Ações				
Ação 1	Faça X	Faça Y	Faça X	Faça Z
Ação 2	Faça A	Faça B	Faça B	Faça B

Grafo de Causa-Efeito

- Exemplo:
 - Embora não seja comum, as Tabelas de Decisão também podem possuir condições com valores complexos (não binários).
 - Entretanto, em geral, sempre é possível convertê-la em uma versão com valores binários.

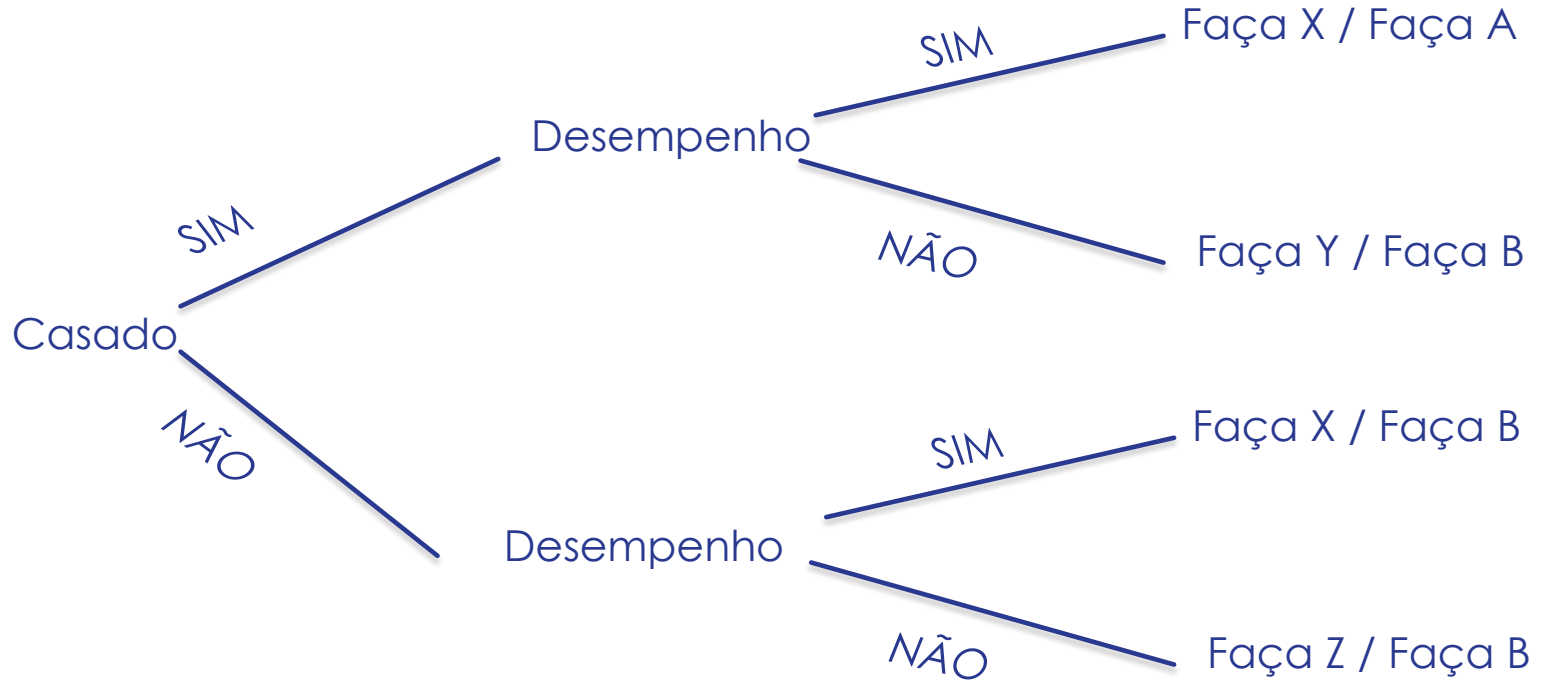
Condições	Entrada 1	Entrada 2	Entrada 3	Entrada 4
Condição 1	0-1	2-10	11-100	101-1000
Condição 2	< 5	5	6 ou 7	> 7
Ações				
Ação 1	Faça X	Faça Y	Faça X	Faça Z
Ação 2	Faça A	Faça B	Faça B	Faça B

Condições	Entrada 1	Entrada 2	Entrada 3	Entrada 4
Condição 1	0-1	2-10	11-100	101-1000
Condição 2	< 5	5	6 ou 7	> 7
Ações				
Ação 1	Faça X	Faça Y	Faça X	Faça Z
Ação 2	Faça A	Faça B	Faça B	Faça B

- Obtida a Tabela de Decisão, cada coluna dará origem a um caso de teste:
 - Condições representam os valores de entrada
 - Ações representam as saídas esperadas

Casos de Teste	Condição 1	Condição 2	Saída
CT ₁	0	3	Faça X / Faça A
CT ₂	5	5	Faça Y / Faça B
CT ₃	50	7	Faça X / Faça B
CT ₄	500	10	Faça Z / Faça B

Árvore de Decisão



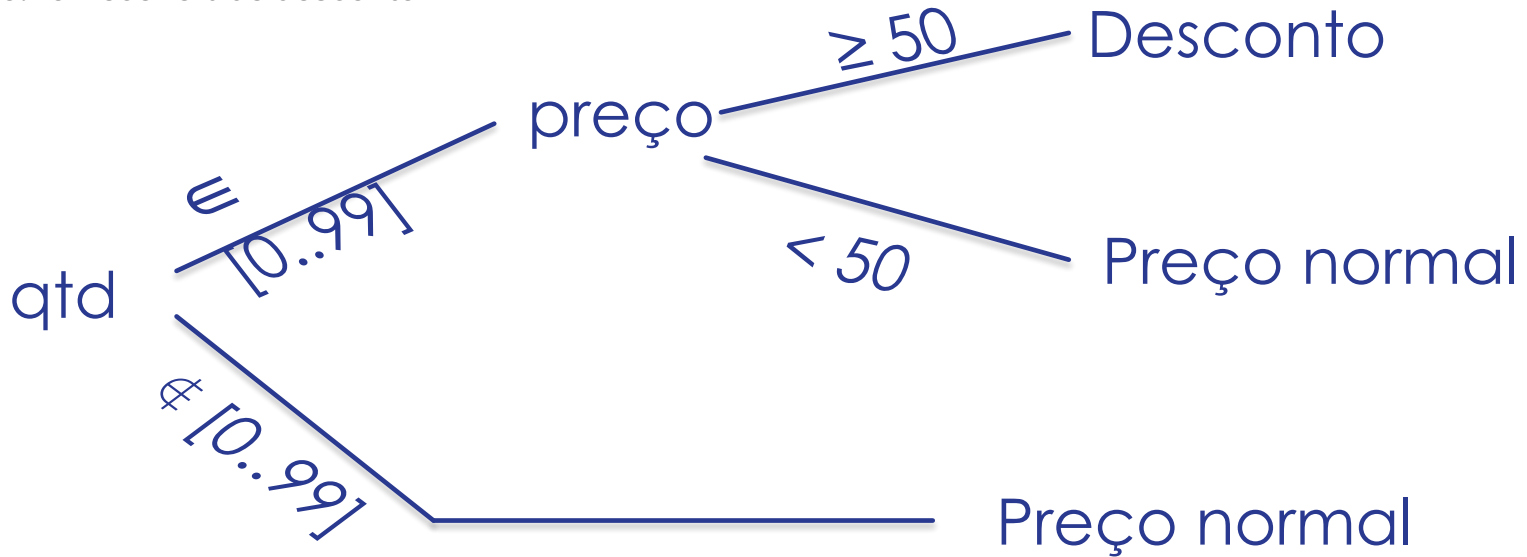
Geração de Testes

- Tabela de decisão
 - critério: exercitar cada regra pelo menos 1 vez
- Árvore de decisão
 - critério: exercitar cada caminho da raiz até a folha pelo menos 1 vez
- Limitação:
 - Tamanho: n° de regras = 2^N , onde N é o n° de causas

Árvore de Decisão

Causa: preço ≥ 50 & $0 \leq \text{qtd} \leq 99$

Efeito: fornecer 5% de desconto





Baseada em Erros

Baseada em Erros



Abordagem *ad hoc*

Processo informal e não estruturado baseado na experiência.



Intuição e Experiência

Supõe tipos prováveis de erros com base em vivências anteriores.



Definição de Casos de Teste

Define CTs focados em detectar os erros identificados.

1. Analisar a especificação de requisitos;
2. Identificar a partir da especificação entradas válidas e inválidas para o sistema;
3. Determinar as saídas esperadas para cada conjunto de entrada;
4. Produzir e documentar o caso de teste;
5. Executar o caso de teste conforme projetado;
6. Capturar as saídas produzidas pelo software e comparar com as saídas esperadas;
7. Documentar a comparação entre o resultado esperado e o produzido

SUMMARY



Vamos de atividade - Mas antes um diagnóstico

1. Qual das opções abaixo representa um valor limite válido para o seguinte requisito?

“O cliente pode realizar pedidos entre R\$10 e R\$500.”

- A) R\$5
- B) R\$10
- C) R\$250
- D) R\$505

Vamos de atividade - Mas antes um diagnóstico

2. Qual das opções melhor descreve uma Tabela de Decisão?

- A) Um diagrama que mostra o caminho de execução do código-fonte
- B) Um conjunto de valores aleatórios usados para testar funcionalidades
- C) Um modelo que relaciona condições de entrada com ações de saída
- D) Um gráfico usado para representar desempenho de testes automatizados

Vamos de atividade - Mas antes um diagnóstico

3. Qual a principal diferença entre Partição em Classes de Equivalência e Análise de Valor Limite?

- A) Partição usa entradas inválidas; valor limite usa apenas válidas
- B) Partição foca em média dos valores; valor limite nos extremos
- C) Partição testa todos os valores possíveis; valor limite testa 1 por classe
- D) Partição agrupa valores semelhantes; valor limite foca nas bordas desses grupos

Requisito simulado: "Uma pessoa pode votar se tiver entre 16 e 70 anos, inclusive."

Classes de Equivalência

Válida: 16 a 70 anos

Inválida 1: Menos de 16 anos

Inválida 2: Mais de 70 anos

Requisito simulado: "Uma pessoa pode votar se tiver entre 16 e 70 anos, inclusive."

Caso	Entrada (idade)	Esperado	Justificativa
CT1	15	Inválido	1 abaixo do mínimo
CT2	16	Válido	Exatamente o limite inferior
CT3	17	Válido	1 acima do mínimo
CT4	69	Válido	1 abaixo do limite superior
CT5	70	Válido	Exatamente o limite superior
CT6	71	Inválido	1 acima do limite superior

“Uma pessoa só pode fazer empréstimo se for maior de 18 anos e tiver renda mensal superior a R\$2.000.”

Condições

- **C1:** Idade ≥ 18 ? (Sim ou Não)
- **C2:** Renda $> \text{R\$}2.000$? (Sim ou Não)

Ação Esperada

- **A1:** Empréstimo aprovado
- **A2:** Empréstimo negado

Tabela de decisão

Regra	C1: Idade ≥ 18	C2: Renda > 2000	Ação Esperada
R1	Sim	Sim	A1: Aprovado
R2	Sim	Não	A2: Negado
R3	Não	Sim	A2: Negado
R4	Não	Não	A2: Negado

CTs

CT	Idade	Renda	Resultado Esperado
CT1	25	3000	Aprovado
CT2	25	1500	Negado
CT3	16	2500	Negado
CT4	16	1500	Negado